

智能图像处理实验指导书

编者：王祖林 徐 迈

王鹏波 李胜曦

邓 欣 蒋 铨

北航电子信息工程学院

2026 年 4 月

目 录

目 录	1
第一章 数字图像处理系统	1
1.1 图像采集	1
1.2 图像显示	4
1.3 图像存储	4
1.4 图像处理和分析	4
第二章 Python 编程基础	5
2.1 Python 基础语法	5
2.2 Python 库	6
2.3 Python 运行	7
2.4 Python 的图形用户界面（GUI）	8
第三章 MATLAB 编程基础	10
3.1 MATLAB 2023 开发环境	10
3.2 MATLAB 的主要图形功能	13
3.3 MATLAB 的图形用户界面（GUI）	16
第四章 深度学习框架基础	17
4.1 Pytorch	17
4.2 Tensorflow	21
第五章 上机实验	25
5.1 实验要求	25
5.2 实验内容	26
附录 A: 实验一图像变换实验指南	32
附录 B: 实验二图像增强复原实验指南	36
参考文献	42

第一章 数字图像处理系统

数字图像处理，即用计算机对图像进行处理，主要应用的是计算机图像技术。这包括利用计算机和其它电子设备进行和完成的一系列工作，例如图像的采集、获取、编码、存储和传输，图像的合成和产生，图像的显示和输出，图像的变换、增强、恢复（复原）和重建，图像的分割，目标的检测、表达和描述，特征的提取和测量，序列图像的校正，3-D 景物的重建复原，图像数据库的建立、索引和抽取。图像的分类、表示和识别，图像模型的建立和匹配，图像和场景的解释和理解，以及基于它们的判断决策和行为规划等等。另外，还可包括为完成上述功能而进行的硬件设计及制作等方面的技术。

一个基本的图像（处理和分析）系统构成可由图 1.1 所示。图中各部分都有特定的功能，分别是采集、显示、存储、处理和分析。

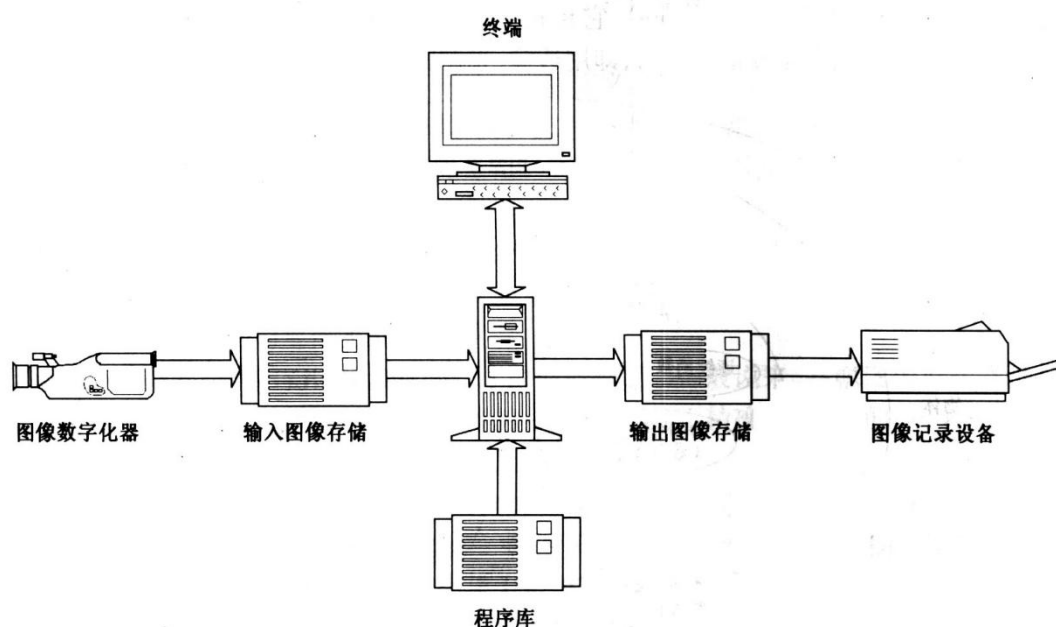


图 1.1 数字图形处理系统

1.1 图像采集

由于计算机只能处理数字图像，为得到采集数字图像，近年来获得发展的图像传感器类型是电子自扫描固态传感器阵列，固态阵是由称为感光基元的离散硅成像元素构成的。这样的感光基元能产生与所接受的输入光强成正比的输出电压。固态阵按几何组织形式分为两种：线扫描器和平面扫描器。线扫描传感器包括一行感光基元，它靠场景和检测器之间的相对运动来获得 2-D 图像。平面扫描传感器由排成方阵的感光基元组成，可直接得到 2-D 图像。固态平面传感器的一个显著特点是它具有非常快的快门速度（可达），所以能将许多运

动定格下来。

固态阵中主要元件是 CCD，CCD 是英文 Charge Coupled Device 即电荷耦合器件的缩写，它是一种特殊半导体器件，上面有很多一样的感光元件，每个感光元件叫一个像素。CCD 摄像机里是一个极其重要的部件，它能够将光线变为电荷并将电荷存储及转移，也可将存储之电荷取出使电压发生变化，是理想的摄像机元件，以其构成的 CCD 摄像机具有体积小、重量轻、灵敏度高、抗强光、畸变小、体积小、寿命长、不受磁场影响、具有抗振动和撞击等优点而被广泛应用。它起到将光线转换成电信号的作用，类似于人的眼睛，因此其性能的好坏将直接影响到摄像机的性能。

图 1.2 给出了一个线扫描 CCD 传感器的示意图。这个传感器由一行感光基元、两个定时地将感光基元中的内容传给传输寄存器的传输门，以及一个定时地将传输寄存器的内容传给放大器的输出门构成。放大器输出的电压信号与感光基元行的内容成比例。

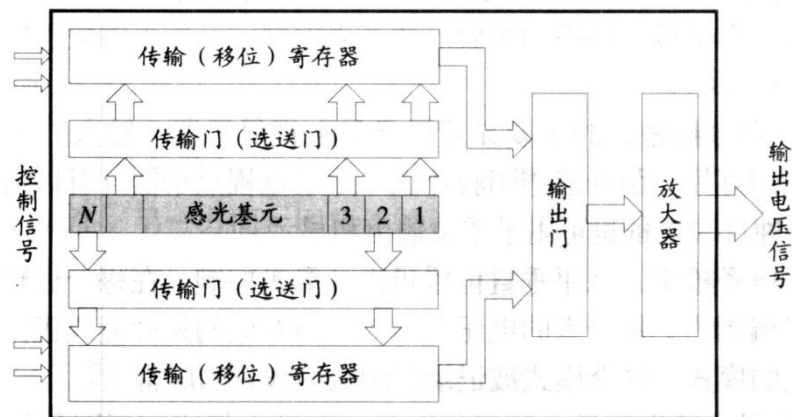


图 1.2 线扫描传感器

电荷耦合平面阵的工作原理与线形相似，但这里感光基元排列成一个矩形形式（如图 1.3 所示）。感光基元列由传输门和传输寄存器隔开。先将奇数列感光基元的内容顺序送进垂直传输寄存器，然后再送进水平传输寄存器。把水平传输寄存器的内容送进放大器就得到 1 帧隔行的视频信号。对偶数感光基元重复以上过程就可得到另 1 帧隔行的视频信号。将两帧合起来就得到隔行扫描电视的 1 场。

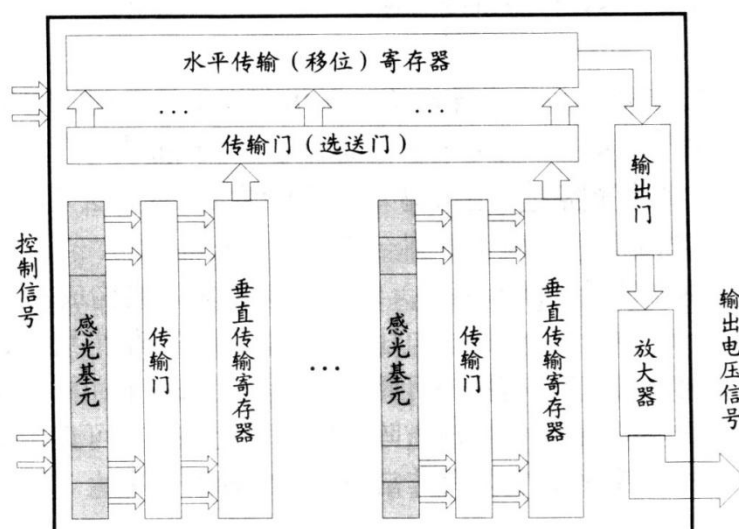


图 1.3 平面扫描传感器

1.1.1. CCD 摄像机特性

1. 电荷耦合器件

CCD 芯片被制造在一片光敏结晶硅片上，一个矩形的光电探测器（势阱）阵列被制造在硅基片中。在局部区域中产生的光电子被收容在最近的势阱中，并作为一个电荷包沿一串势阱移动直到到达外部引出端。

2. CCD 摄像机的工作方式

被摄物体的图像经过镜头聚焦到 CCD 芯片上，CCD 根据光的强弱积累相应比例的电荷，各个像素积累的电荷在视频时序的控制下，逐点外移，经滤波、放大处理后，形成视频信号输出。视频信号连接到监视器或电视机的视频输入端便可以看到与原始图像相同的视频图像。

3. 分辨率的选择

评估摄像机分辨率的指标是水平分辨率，其单位为线对，即成像后可以分辨的黑白线对的数目。常用的黑白摄像机的分辨率一般为 380-480，其数值越大成像越清晰。一般的监视场合，用 400 线左右的黑白摄像机就可以满足要求。而对于医疗、图像处理等特殊场合，用 600 线的摄像机能得到更清晰的图像。

4. 成像灵敏度

通常用最低环境照度要求来表明摄像机的灵敏度，黑白摄像机的灵敏度大多是 0.02-0.5Lux(勒克斯),彩色摄像机多在 1Lux 的摄像机用于普通像机用于普通的监视场合；在夜间使用或环境光线较弱时，推荐使用 0.02Lux 的摄像机。与近红外灯配合使用时，也必须使用低照度的摄像机。另外摄像机的灵敏度还与镜头有关，0.97Lux/F0.75 相当于 2.5Lux/F1.4。

5. 电子快门

电子快门的时间在 1/50-1/100,000 秒之间，摄像机的电子快门一般设置为自动电子快门方式，可根据环境的亮暗自动调节快门时间，得到清晰的图像。

有些摄像机允许用户自行手动调节快门时间，以适应某些特殊应用场合。

6. 外同步与外触发

外同步是指不同的视频设备之间用同一同步信号来保证视频信号的同步，它可保证不同的设备输出的视频信号具有相同的帧、行的起止时间。为了实现外同步，需要给摄像机输入一个复合同步信号（C-sync）或复合视频信号。外同步并不能保证用户从指定时刻得到完整的连续的一帧图像，要实现这种功能，必须使用一些特殊的具有外触发功能的摄像机，如敏通公司的 MS-2821 等。

7. 光谱响应特性

CCD 器件由硅材料制成，对近红外比较敏感，光谱响应可延伸至 1.0 μm 左右。其响应峰值为绿光（550nm）。夜间隐蔽监视时，可以用近红外灯照明，人眼看不清环境情况，在监视器上却可以清晰成像。由于 CCD 传感器表面有一层吸收紫外的透明电极，所以 CCD 对紫外不敏感。彩色摄像机的成像单元上有红、绿、蓝三色滤光条，所以彩色摄像机对红外、紫外均不敏感。

8. CCD 芯片的尺寸

CCD 的成像尺寸常用有 1/2, 1/3 等，成像尺寸越小的摄像机的体积可以做得更小些。在相同的光学镜头下，成像尺寸越大，视通角越大。

1.2 图像显示

对图像处理来说，处理的结果主要用来显示给人看。对图像分析来说，分析的结果也可以借助计算机图形学技术转换为图像形式直观的展示。所以图像显示对图像处理和分析系统来说是非常重要的。

常用的图像处理和分析系统的主要显示设备是电视显示器。最重要的显示特性是图像的大小、光度分辨率和空间分辨率、低频响应和噪声特性。

1.3 图像存储

图像包含有大量的信息，因而存储图像也需要大量的空间。在图形处理和分析系统中，大容量和快速的图像存储器是必不可少的。用于图像处理和分析的数字存储器可分为 3 类：

- 1、处理和分析过程中使用的快速存储器；
- 2、用于比较快地重新调用的在线或联机存储器；
- 3、不经常使用的数据库（档案库）存储器。

1.4 图像处理和分析

对图像的处理和分析一般可用算法的形式描述，而大多数的算法可用软件实现，只是在为了提高速度或克服通用计算机限制的情况下才用特制的硬件。

图像处理和分析中的一个重要事实是对特殊的问题需要特殊的解决方法。现有的图像处理和分析软件和硬件可以提供比以前更多更快地通用工具，但要解决具体的问题还需要更深入的研究和开发。

第二章 Python 编程基础

Python 是一种面向对象的解释型语言，Python 语法简洁清晰，强制用空白符(white space)作为语句缩进。其最大特点就是简单、强大。Python 具有丰富强大的库，常被称为胶水语言，能够把用其他语言制作的各种模块很轻松地联结在一起。

2.1 Python 基础语法

2.1.1. Python 标识符

在 Python 里，标识符由字母、数字、下划线组成。

在 Python 中，所有标识符可以包括英文、数字以及下划线(_)，但不能以数字开头。

Python 中的标识符是区分大小写的。

以下划线开头的标识符是有特殊意义的。以单下划线开头_foo 的代表不能直接访问的类属性，需通过类提供的接口进行访问，不能用 from xxx import * 而导入。

以双下划线开头的__foo 代表类的私有成员，以双下划线开头和结尾的__foo__ 代表 Python 中特殊方法专用的标识，如__init__()代表类的构造函数。

Python 可以同一行显示多条语句，方法是用分号;分开。

2.1.2. Python 保留字符

所有 Python 的关键字只包含小写字母。

and	exec	not
assert	finally	or
break	for	pass
class	from	print
continue	global	raise
def	if	return
del	import	try
elif	in	while
else	is	with
except	lambda	yield

2.1.3. 行和缩进

Python 的代码块不使用大括号{}来控制类,函数以及其他逻辑判断。Python 最具特色的就是用缩进来写模块。缩进的空白数量是可变的,但是所有代码块语句必须包含相同的缩进空白数量,这个必须严格执行,否则执行代码时会报错。

所以在 Python 的代码块中必须使用相同数目的行首缩进空格数。建议每个缩进层次使用单个制表符或两个空格或四个空格,切记不能混用。

2.1.4. 多行语句

Python 语句中一般以新行作为语句的结束符。但是我们可以使用斜杠 (\) 将一行的语句分为多行显示。

2.1.5. Python 引号

Python 可以使用引号(')、双引号(")、三引号('' 或 ''') 来表示字符串,引号的开始与结束必须的相同类型的。其中三引号可以由多行组成,编写多行文本的快捷语法。

2.2 Python 库

2.2.1. PIL 和 OpenCV

PIL (Python Imaging Library) 和 OpenCV (OpenSource Computer Vision) 是使用 Python 语言进行图像处理时常用到的两个库, PIL 的调用示例如下:

```
from PIL import Image, ImageFilter
#读取图像
im = Image.open( 'image.jpg' )
#显示图像
im.show()
```

OpenCV 在图像操作与处理上相对 PIL 更为先进且使用得更加广泛,其调用示例如下:

```
import cv2
#读取图像
img = cv2.imread('testing.jpg')
#显示图像
cv2.imshow('image',img)
cv2.waitKey(0)
cv2.destroyAllWindows()
```

imread()是读取图像的函数,被读入的图像以 ndarray 格式的数组存储,取值范围为[0, 255];

imshow()是显示图像的函数,需要与 waitKey(delay)结合使用,参数 delay

表示延迟的时间，单位为毫秒，默认值为 0，而调用 `destroyAllWindows()` 函数可以关闭由 OpenCV 创建的所有窗口；

`imwrite()` 是存储图像的函数，第一个参数是图像的命名，第二个参数是要存储的图像对象。

2.2.2. 其他常用库

Numpy: 为 Python 提供快速的多维数组处理能力，与 `cv2` 配合使用，通常的用法为 `import numpy as np`，这样在调用时更加简便；数组统计时常用到的函数有：

函数	用途
<code>sum()</code>	对数组中全部或某轴向的元素求和
<code>mean()</code>	算数平均数
<code>std()</code>	计算标准差
<code>var()</code>	计算方差
<code>min()</code>	求最小值
<code>max()</code>	求最大值

Scipy: 在 NumPy 基础上添加了众多科学计算工具包

Matplotlib: Python 丰富的绘图库，可以用来绘制图像，相当于 MATLAB 中的 `plot`，可设置多个 `figure` 以及 `figure` 的标题，也可以使用 `subplot` 在一个 `figure` 中显示多个图像，调用示例如下：

```
import matplotlib.pyplot as plt
```

2.3 Python 运行

有三种方式可以运行 Python：

2.3.1. 交互式解释器：

可以通过命令行窗口进入 `python` 并开在交互式解释器中开始编写 Python 代码。可以在 Unix, DOS 或任何其他提供了命令行或者 `shell` 的系统进行 `python` 编码工作。

```
$ python # Unix/Linux  
或者  
C:>python # Windows/DOS
```

2.3.2. 命令行脚本

在你的应用程序中通过引入解释器可以在命令行中执行 Python 脚本，如下所示：

```
$ python script.py # Unix/Linux  
或者  
C:>python script.py # Windows/DOS
```

2.3.3. 集成开发环境（IDE）：PyCharm

PyCharm 是由 JetBrains 打造的一款 Python IDE，使用广泛，支持 macOS、Windows、Linux 系统。

PyCharm 功能：调试、语法高亮、Project 管理、代码跳转、智能提示、自动完成、单元测试、版本控制等

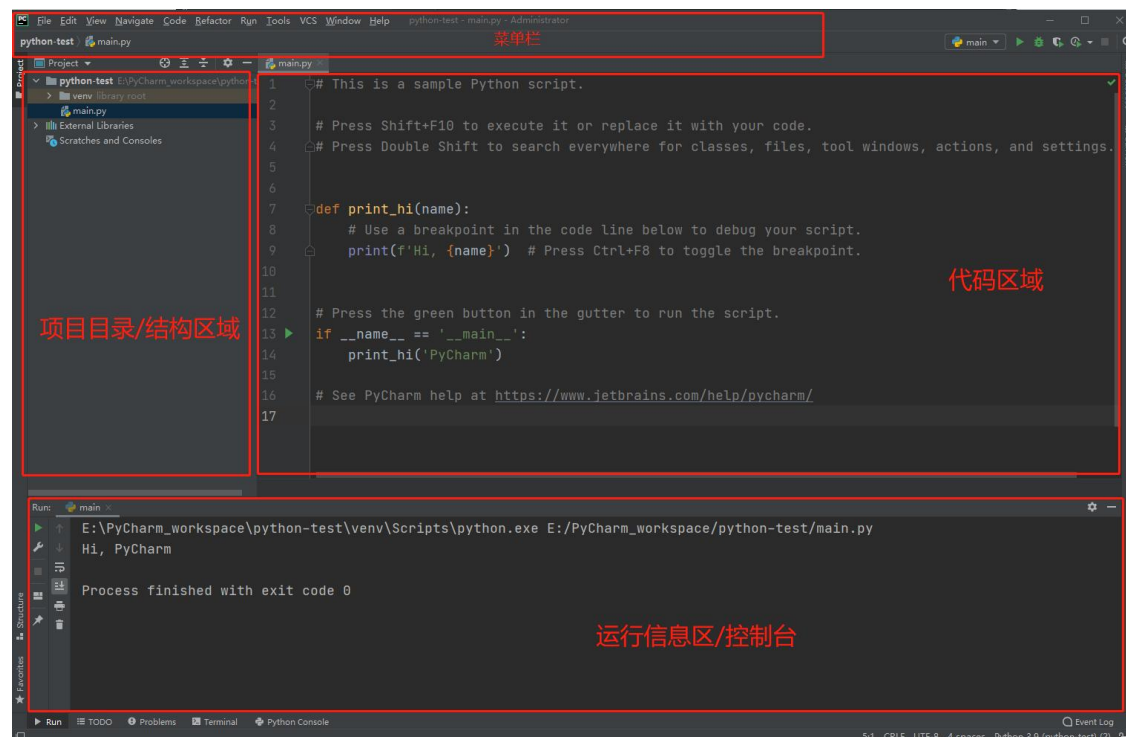


图 2.1 Pycharm 编程界面示意图

2.4 Python 的图形用户界面（GUI）

2.4.1. 如何创建一个 GUI 用户界面窗口？

开发可交互图像界面 QT 主要需用两个工具包：PySide2 或 PyQt5。我们以 PyQt5 为例，可以通过指令进行下载 PyQt5：

```
pip install PyQt5
```

安装 QT 的工具包后，可以通过编程实现图像用户界面，下面是实现一个基础 GUI 界面的编程过程。

首先需要导入相关类：

```
1. import sys
2. from PyQt5.QtWidgets import QApplication, QWidget, QLabel, QPushButton
```

再导入相关类后，需要创建一个对象，用于管理图像用户界面：

```
1. app = QApplication(sys.argv)
```

现在可以通过 `window = QWidget()` 创建第一个窗口界面 `window`。
如果需要设置窗口大小，可以调用函数 `window.resize(width, height)`。
如果需要设置窗口标题，可以调用函数 `window.setWindowTitle('标题')`。

窗口创建后可以在窗口上创建文本空间、按钮等组件。

如果需要创建标签来显示不可编辑的文本或图片，可以调用函数：

```
2. label = QLabel()
```

如果需要创建按钮，可以调用函数：

```
3. button = QPushButton('name', window)
```

同样地，可以通过函数 `button.move(width, height)` 设置按钮在窗口的位置。

最后需要加入以下两行代码，实现界面显示：

```
4. window.show()
```

```
5. sys.exit(app.exec_())
```

至此，一个简单的 GUI 用户界面窗口就完成了

2.4.2. 在用户界面窗口里添加一个按钮，并显示 LENA 图像：

步骤 1：按照 1 中步骤创建一个对象与用户界面窗口。

步骤 2：通过代码设置一个按钮组件：

```
1. self.button = QPushButton("按钮", window)
```

步骤 3：编写函数实现显示 LENA 图像事件：

`def btn_clicked(self):`

```
2. pixmap = QPixmap('lena.png')
```

```
3. label = QLabel()
```

```
4. label.setPixmap(pixmap)
```

```
5. label.show()
```

步骤 4：把按钮点击与图像显示事件绑定在一起。

```
6. self.button.clicked.connect(self.clickButton)
```

按上述实现简单示例的完整代码如下：

```
1. import sys
```

```
2. from PyQt5.QtGui import QPixmap
```

```
3. from PyQt5.QtWidgets import QMainWindow, QApplication, QPushButton, QLabel
```

```
4. class MainWindow(QMainWindow):
```

```
5.     def __init__(self, parent=None):
```

```
6.         super(MainWindow, self).__init__(parent)
```

```
7.         self.resize(280, 330)
```

```
8.         self.setWindowTitle('创建按钮和按钮点击事件的例子')
```

```
9.         self.label = QLabel(self)
```

```
10.        self.label.move(0, 60)
```

```

11.         self.label.resize(256, 256)
12.         self.button1 = QPushButton('按钮 1', self)
13.         self.button1.move(0,0)
14.         self.button1.resize(100,50)
15.         self.button1.clicked.connect(self.clickButton)
16.         def clickButton(self):
17.             pixmap = QPixmap('lena.bmp')
18.             self.label.setPixmap(pixmap)
19.             self.label.show()
20.         if __name__ == '__main__':
21.             app = QApplication(sys.argv)
22.             main = MainWindow()
23.             main.show()
24.             sys.exit(app.exec_())

```

当然，除去上述复杂的编程实现外，也可以通过 QT designer 实现界面的布局，对 GUI 变成进行简化，同时也可以实现更加复杂的 GUI 界面。

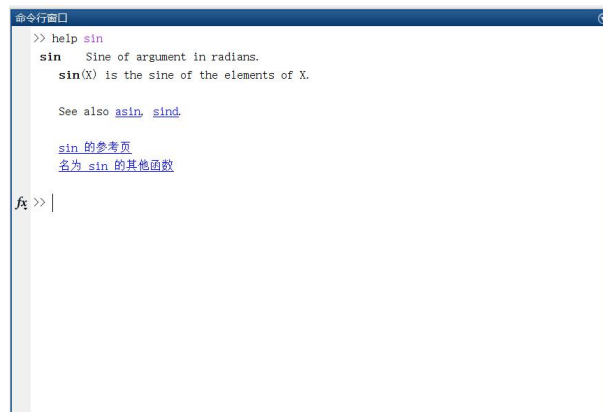
第三章 MATLAB 编程基础

Matlab 软件包由主包、Simulink 以及功能各异的工具箱组成，它以矩阵运算为基础，把计算、课时化、程序设计融合到了一个简单易用的交互式工作环境中。在这里可以实现工程设计、算法研究、符号运算、建模与仿真、原型开发、数据分析既可视化、科学和工程绘图、应用程序设计（包括图形用户界面设计）等功能。

3.1 MATLAB 2023 开发环境

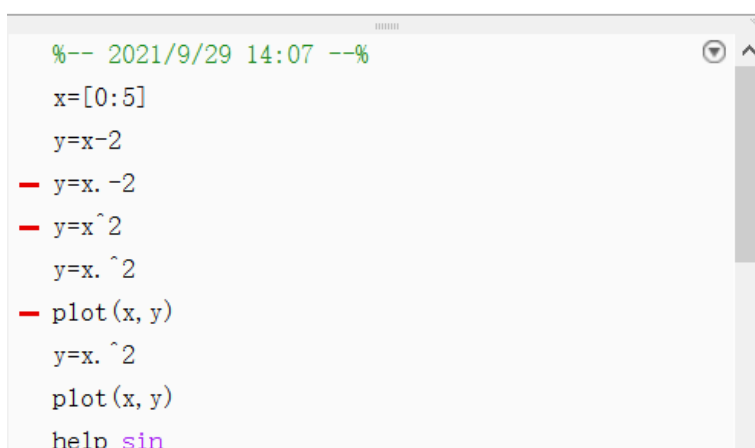
MATLAB 2023 的默认开发环境包括以下部分：

3.1.1. Command Window（命令）窗口



命令窗口是用户和 MATLAB 交互的主要方式，用户可以在 Command Window 窗口中运行函数和执行 MATLAB 操作。

3.1.2. Command History (命令历史) 窗口

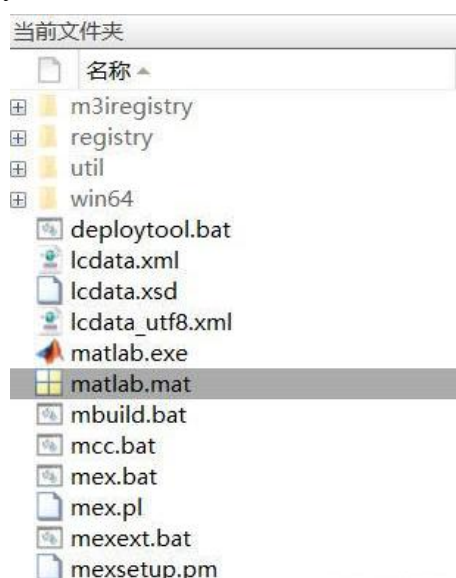


```
%-- 2021/9/29 14:07 --%
x=[0:5]
y=x-2
- y=x.^2
- y=x.^2
y=x.^2
- plot(x,y)
y=x.^2
plot(x,y)
help sin
```

用户可以在此窗口里查看曾经在 Command Window 窗口里输入的命令, 并可在 Command Window 窗口里运行他们。运行 Command History 窗口里的命令有如下几种方式:

- ✓ 在 Command Window 窗口里, 利用键盘上的上下移动键选定 Command History 窗口里的命令, 然后按 Enter 键。
- ✓ 用鼠标选定它们, 然后复制到 Command Window 窗口, 最后按 Enter 键。
- ✓ 用鼠标在 Command History 窗口里双击要执行的命令。

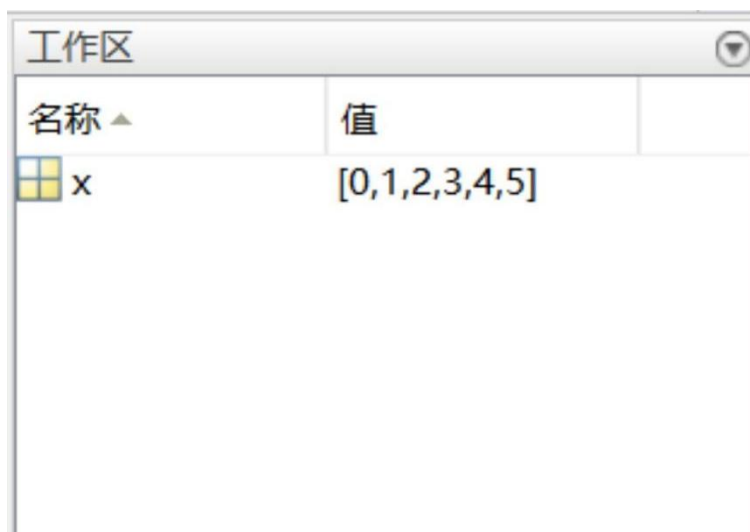
3.1.3. Current Directory (当前目录) 浏览器



在此可以查看 MATLAB 文件和与 MATLAB 有关的文件, 并可以进行一些文件操作。在这里能够进行的主要操作如下:

- ✓ 查看和改变目录
- ✓ 创建、重命名、复制和移动文件夹或文件
- ✓ 打开、运行文件和查看文件的内容

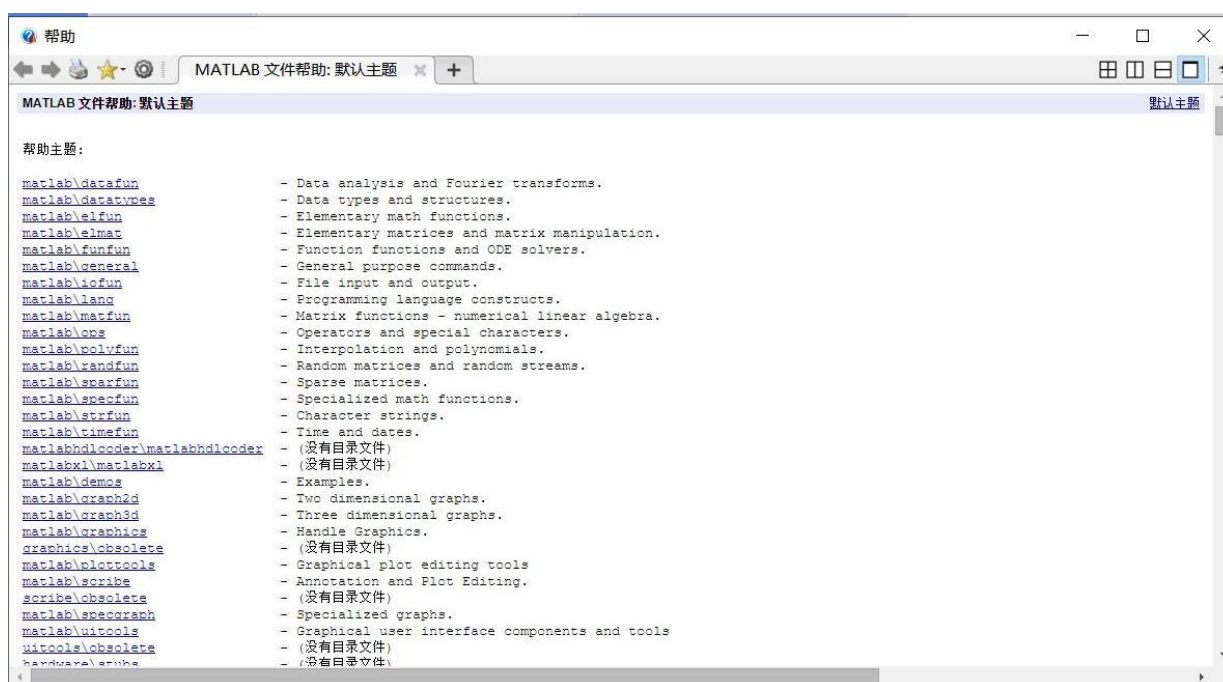
- ✓ 查找和替换文件的内容
- ✓ Workspace（工作窗口）浏览器



用户可以在此查看和改变工作空间的内容。在这里能够进行的主要操作如下：

- ✓ 查看当前的工作空间
- ✓ 保存当前的工作空间
- ✓ 装载一个保存好的工作空间
- ✓ 清除工作空间的变量
- ✓ 查看基本函数的工作空间

3.1.4. Help Window(帮助)窗口



是用户可以在此窗口获得 MATLAB 的官方辅助文档。

3.2 MATLAB 的主要图形功能

3.2.1. 二位图像绘制函数

函数	用途
plot(Y)	如果 Y 是向量，并且其元素是实数，则绘制出的是 Y 与其元素的下标所构成的二元组的曲线图形，这等价于 plot(1:length(Y),Y)。若 Y 的元素是复数，则 plot(Y)等价于 plot(real(Y), imag(Y))，其中 real(Y)是 Y 的实部、imag(Y)是 Y 的虚部。若 Y 是矩阵，就按列绘制曲线图，曲线的条数等于 Y 矩阵的列数。
plot(X1,Y1,.....)	以(X1,Y1), (X2,Y2), (X3,Y3)...为二元组，绘制多条曲线。
plot(X1,Y1,LineSpec,...)	LineSpec 是线的定义，包括线性、标记符号、颜色。
axis([xmin,xmax,ymin,ymax])	指定坐标轴的范围，其中[xmin,xmax]是 x 坐标轴的范围，[ymin,ymax]是 y 坐标轴的范围。
axis auto	使坐标轴的范围返回到原来的默认模式。
title('string')	在当前坐标系的顶部加一个文本串，作为该图形的标题。
Xlabel('string')	在当前坐标系 X 轴的旁边加一个文本串，作为 X 轴的名称。
Ylabel('string')	在当前坐标系 Y 轴的旁边加一个文本串，作为 Y 轴的名称。
grid on	在当前坐标系中附加网格线。
grid off	在当前坐标系中去掉网格线。
legend('string1','string2',..., pos)	其中 string1, string2,...是用户指定的说明字符串，依次对图形中的各种成分(包括线图、条形图等)进行说明。pos 为一整数，指定图例的位置。
legend('off')	删除图例说明框。

3.2.2. 显示位图的基本函数

函数	用途	功能组
Axis	设置坐标轴的比例和外观	显示
Image	显示图象（创建图像对象）	显示
Imagesc	比例化数据并显示成图像	显示
Imread	从图像文件里读出图像	文件输入/输出
Imwrite	把图像写到图形文件里	文件输入/输出
Imfinfo	从图形文件里获取图像信息	应用函数

ind2rgb	把索引图像转化成 RGB 图像	应用函数
----------------	-----------------	------

3.2.3. 图像的种类

a.索引图像：一个索引图像包含一个数据矩阵 X 和一个色图矩阵 map 。其中 map 是一个 $m \times 3$ 的双精度数组，值的范围是 $[0,1]$ 。可以用以下两个函数来显示索引图像。

image(X) 产生包含图像数据的矩阵

colormap(map) 产生由该图像所给的特定色图

例：

```
clear;
close all;
load earth;
image(X);
colormap(map);
axis equal;
axis off;
title( 'The Earth' );
```

b.灰度图像：由一个数据矩阵存储，它的每个元素对应着图像的一个像素，它的元素值表示一定范围内的灰度值。灰度图像很少和色图矩阵一起保存，一般使用系统提供的色图函数来产生。本质上 MATLAB 也是按索引图像来处理灰度图像的。可以用 `imagesc` 函数像是一个灰度图像，用户可以设置灰度值的范围，例如：

```
imagesc(I,[0 1]);
colormap(gray);
```

c. 真彩（RGB）图像：一般存储在一个 $m \times n \times 3$ 的数组里，分别定义了红、绿和蓝 3 个颜色分量的值。真彩图像虽然不需要色图矩阵，但是要以牺牲大量的存储空间为代价。

例：由一个索引图像生成一个真彩图像的数组并显示

```
load earth;
RGB=ones(size(X,1),size(X,2),3);
for i=1:size(X,1)
for j=1:size(X,2)
RGB(i,j,:)=map(X(i,j,:),:);
end
end
image(RGB);
axis image;
```


3.2.4. 图像文件的读写和查询

imread(filename) 读取图像文件

例: clear;
RGB=imread('first.jpg');
image(RGB);

imwrite(X,map,filename) 写图像文件

例: load flower;
imwrite(X,map,'flower.bmp');

就可以根据索引图像数据 X 和色图 map 生成一个 bmp 图像文件。

info=imfinfo(filename) 查询图像文件信息

3.2.5. 三维图形

函数	用途
plot3(X1,Y1,Z1,...)	如果 X1,Y1,Z1,...是向量, 则根据向量 X1,Y1,Z1,...绘制空间三维曲线。X1,Y1,Z1,...为同规模的向量, 三者对应的元素构成曲线上点的坐标; 如果 X1,Y1,Z1,...是矩阵, 则绘制对应列的多条曲线。
plot3(X1,Y1,Z1,LimeSpec1,...)	(X1,Y1,Z1),(X2,Y2,Z2),...为三元组, 分别按字符串 LimeSpec1、LimeSpec2...制定的线性、颜色、和点的标记来绘制多条曲线。其中 LimeSpec 的用法和绘制二维曲线图形时的用法一样。
fill3(X,Y,Z,C)	绘制由向量 X,Y 和 Z 定义的多边形。C 为字符串, 制定填充颜色。
fill3(X1,Y1,Z1,C1,X2,Y2,Z2,C2,...)	绘制有多组参数指定的多个多边形, 并按对应的颜色进行填充。

3.2.6. 特殊坐标

函数	用途
loglog	使用双对数坐标作图。X 和 Y 的两个坐标采用对数刻度。
plotyy	使用双 Y 坐标作图。
polar	绘制极坐标图。
semilogx	只是 X 轴采用了对数坐标, Y 轴仍采用对数刻度。
semilogy	只是 Y 轴采用了对数坐标, X 轴仍采用对数刻度。

3.2.7. 条形图和区域图

函数	用途
area	填充二维区域。
fill	对多边形进行填充。

bar	绘制垂直条形图。
barh	绘制水平条形图。
bar3	绘制垂直三维条形图。
bar3h	绘制水平三维条形图。

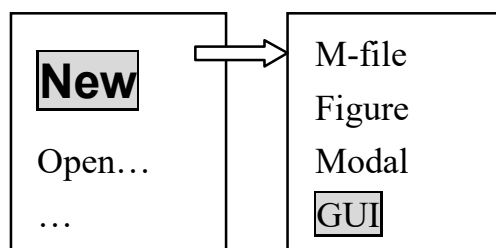
备注：各种函数可以通过 Help 的查询得到更详细的介绍和示例，也可通过 Help->Demos 了解 Matlab 的函数功能及应用。

3.3 MATLAB 的图形用户界面（GUI）

3.3.1. 如何创建一个 GUI 用户界面窗口？

启动 MATLAB 软件后，进入集成开发环境，在主菜单上选择：

File Edit Web Window Help

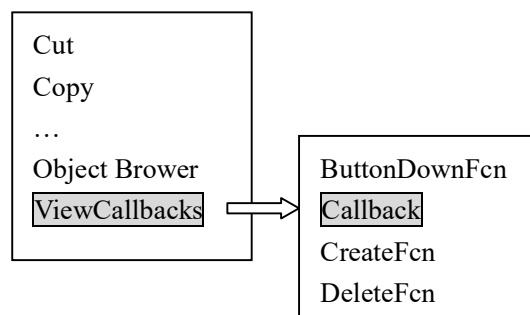


如上操作后，将有一个窗口弹出，即进入 MATLAB 的 GUI 编辑软件。利用该编辑软件的“New-新建”创建你自己的用户界面窗口。注意：要编辑一个以前设计的用户界面，操作过程与此类似，进入 GUI 编辑软件后，利用“Open-打开”来选择要编辑的以前设计的用户窗口界面。

3.3.2. 在用户界面窗口里添加一个按钮，并显示 LENA 图像：

步骤 1：向用户窗口界面里添加一个按钮。

步骤 2：将鼠标移到按钮上，点击右键，将会弹出一个菜单，做如下选择：



步骤 3：在上一步做完后，如果是新创建用户界面窗口，计算机将提示你输入一个文件名，请在字母 A 后加上你的学号作为你创建的用户界面窗口的文件名，随后将出现一个 M 文件的编辑窗口。该 M 文件是与用户界面窗口相关的一个文件，文件名的后缀不一样。在 M 文件的编辑窗口中，光标将定位在一个反显的字符行上，在该语句后，输入以下两条语句：

```
function varargout = pushbutton1_Callback(h,eventdata,handles,varargin)
B=imread('lean.bmp');
Imshow(B)
```

步骤 4: 把该 M 文件存起来, 回到 MATLAB 的 command 窗口。

步骤 5: 运行你所编辑的 M 文件。此后将出现你所设计的用户界面窗口。

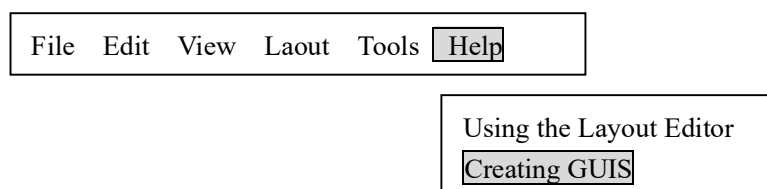
步骤 6: 点击用户界面中的按钮, 将能看到 LENA 的图像。

步骤 7: 此后, 在这个用户界面窗口程序中加入本实验所要求做的其他内容。

注意: lean.bmp 这个文件要与你所编辑的 M 文件位于同一个目录下。

3.3.3. 如何获得关于 GUI 用户界面窗口的更进一步的信息?

进入 GUI 编辑软件, 选择:



详细阅读该文档。

第四章 深度学习框架基础

深度学习框架指专门用于构建、训练、验证与部署深度神经网络的一体化开发工具集, 是整合张量运算、自动求导、网络层封装、硬件加速等核心功能, 降低深度学习算法开发难度的基础环境包。常见的深度学习框架包括 PyTorch、TensorFlow、飞桨 (PaddlePaddle) 和昇思 (MindSpore) 等。其中 TensorFlow 在 2015 年由谷歌开源推出, 因工业生态完善、静态图部署优势突出而迅速普及并占据行业主流地位, 近年来, PyTorch 依托动态图灵活调试、语法简洁易懂、社区生态活跃的优势, 成为科研领域的新的主流选用框架。本章将以 PyTorch 和 TensorFlow 为例, 讲解深度学习框架的核心机制、基础语法、张量操作、自动微分与模型搭建流程, 掌握两大主流框架的核心使用方法。

4.1 Pytorch

PyTorch 是由 Meta (原 Facebook) 人工智能团队研发并开源的轻量级深度学习框架, 基于传统 Torch 机器学习框架重构优化而来, 完全基于 Python 语言开发, 是面向深度学习实验、算法创新、科研迭代的专用框架。相较于传统框架, PyTorch 摒弃了繁琐的静态图机制, 采用动态计算图核心架构, 具备上手简单、调试便捷、自定义程度高的特点, 是目前高校深度学习课程实验、学术论文算法复现、新型网络模型研发的首选框架。

4.1.1. Pytorch 概述

PyTorch 于 2017 年正式开源发布，自推出以来持续迭代更新，不断完善功能体系与硬件适配能力。该框架覆盖深度学习全流程实验操作，核心功能包含多维张量运算、自动梯度求解、神经网络层封装、数据集预处理、模型训练优化、模型保存与加载、硬件加速运算等。

在实验适配性方面，PyTorch 原生支持 CPU、GPU 加速运算，同时兼容国产 AI 算力芯片、服务器集群等多类硬件平台，可满足单机小样本实验、大规模数据集训练等不同实验场景。同时配套 TorchVision、TorchText、TorchAudio 等官方拓展工具库，分别适配计算机视觉、自然语言处理、语音识别等细分实验方向，能够快速支撑各类基础深度学习实验与进阶算法实验，具备极强的实验通用性。

4.1.2. Pytorch 的常见用法

PyTorch 采用模块化设计，以下常用的核心功能及用法，也是本课程实验的中将用到的操作内容：

1. 核心张量运算（torch 基础模块）

张量（Tensor）是 PyTorch 的基础运算单元，也是深度学习实验的数据载体，所有网络运算、数据计算均基于张量完成。该基础模块主要用于实验数据的定义、创建、转换与基础运算。实验中可通过该模块创建常量张量、随机张量、零张量、单位张量，同时支持张量的加减乘除、矩阵运算、维度变换、数据类型转换、CPU/GPU 设备迁移等操作。该用法是所有深度学习实验的基础，主要用于实验数据初始化、数据集预处理、运算数据调试等场景。

```
1. import torch
2. # 1. 创建随机张量（常用网络输入数据）
3. x = torch.randn(32, 3, 32, 32) # 批次 32、3 通道、32*32 图像张量
4. # 2. 创建全零张量
5. zero_tensor = torch.zeros(64, 128)
6. # 3. 张量维度变换
7. reshape_x = x.reshape(32, -1)
8. # 4. 张量 GPU 设备迁移
9. if torch.cuda.is_available():
10.     x = x.cuda()
```

2. 自动微分计算（torch.autograd 模块）

自动微分是深度学习模型训练的核心核心机制，也是 PyTorch 的核心优势模块。该模块完全封装了反向传播的梯度计算逻辑，无需实验人员手动推导复杂的链式求导公式。在实验操作中，只需对需要更新的模型参数开启梯度追踪，

前向运算完成后，通过内置函数即可自动完成损失梯度的求解、累积与反向传播，快速实现模型参数的迭代更新，广泛应用于所有神经网络的训练实验中。

```
1. import torch
2. # 1. 创建可追踪梯度的张量
3. w = torch.tensor([1.0, 2.0, 3.0], requires_grad=True)
4. b = torch.tensor([0.5], requires_grad=True)
5. # 2. 前向传播计算
6. y = torch.sum(w * 2 + b)
7. # 3. 自动反向传播求梯度
8. y.backward()
9. # 4. 查看梯度结果
10. print(w.grad)
11. print(b.grad)
```

3. 神经网络模型搭建 (torch.nn 模块)

torch.nn 是搭建神经网络模型的核心实验模块，封装了深度学习中所有通用的网络层、损失函数与模型结构。实验中可直接调用该模块快速构建各类神经网络：包含全连接层、卷积层、池化层、循环神经网络层、归一化层、激活函数层等基础网络结构；同时内置交叉熵损失、均方误差损失等各类常用损失函数。实验人员可通过该模块堆叠网络层、定义前向传播逻辑，快速搭建分类网络、回归网络、卷积网络等完整模型，大幅简化模型搭建流程。

```
1. import torch
2. import torch.nn as nn
3. # 1. 定义二维卷积层（实验最常用）
4. # 参数：输入通道 3、输出通道 16、卷积核 3*3、步长 1、填充 1
5. conv2d = nn.Conv2d(in_channels=3, out_channels=16, kernel_size=3, stride=1, padding=1)
6. # 2. 定义最大池化层
7. pool = nn.MaxPool2d(kernel_size=2, stride=2)
8. # 3. 定义全连接层
9. fc = nn.Linear(16 * 16 * 16, 10)
10. # 4. 常用激活函数
11. relu = nn.ReLU()
12. # 5. 常用损失函数
13. ce_loss = nn.CrossEntropyLoss() # 分类任务损失
14. mse_loss = nn.MSELoss() # 回归任务损失
```

4. 数据集加载与预处理 (torch.utils.data 模块)

该模块是深度学习数据实验的核心工具，主要解决实验数据集的读取、划分、批量加载问题。模块内置 Dataset 数据集抽象类与 DataLoader 数据加载器，

实验中可自定义数据集读取规则，适配本地自定义数据集，也可结合 TorchVision 调用官方公开数据集(如 MNIST 手写数据集、CIFAR 图像数据集)。同时支持数据打乱、批量分组、多线程加载等功能，能够有效解决实验中数据加载慢、数据无序、批量训练繁琐的问题，是模型训练实验的必备操作。

```
1. import torch
2. from torch.utils.data import DataLoader
3. from torchvision import datasets, transforms
4. # 1. 定义数据预处理方式
5. transform = transforms.Compose([
6.     transforms.ToTensor(),
7.     transforms.Normalize((0.1307,), (0.3081,))])
8. # 2. 加载官方 MNIST 手写数据集
9. train_dataset = datasets.MNIST(root='./data', train=True, transform=transform)
10. # 3. 批量加载数据 (批量大小 64、打乱数据、多线程加载)
11. train_loader = DataLoader(
12.     dataset=train_dataset,
13.     batch_size=64,
14.     shuffle=True,
15.     num_workers=2)
```

5. 模型参数优化 (torch.optim 模块)

模型优化器模块用于实验中模型参数的迭代更新与优化,是提升模型精度、降低损失值的核心模块。该模块内置深度学习实验中所有主流优化器,包含 SGD 随机梯度下降、Adam 自适应优化器、RMSprop、Adagrad 等。实验中只需传入网络模型参数与自定义学习率,即可完成优化器初始化,配合梯度反向传播,实现模型参数的自动迭代优化,完成模型训练与收敛调优。

```
1. import torch.optim as optim
2. # 假设 model 为自定义神经网络模型
3. model = nn.Sequential(
4.     nn.Conv2d(3, 16, 3, 1, 1),
5.     nn.ReLU(),
6.     nn.Flatten(),
7.     nn.Linear(16*32*32, 10))
8. # 1. 定义 Adam 自适应优化器 (实验首选)
9. adam_optim = optim.Adam(model.parameters(), lr=1e-3)
10. # 2. 定义 SGD 随机梯度下降优化器
11. sgd_optim = optim.SGD(model.parameters(), lr=1e-2, momentum=0.9)
12. # 3. 梯度清零 (每次迭代必备)
13. adam_optim.zero_grad()
14. # 4. 参数更新
15. adam_optim.step()
```

4.1.3. Pytorch 的特点分析

结合深度学习实验实操场景，PyTorch 相较于其他框架，具备极强的实验适配性，核心特点如下：

(1) 动态计算图，调试便捷，适配实验迭代：PyTorch 采用动态计算图机制，代码运行过程中实时构建计算图，运算逻辑可随时修改、中断、调试。在实验过程中，可逐行查看张量运算结果、梯度变化情况，快速定位代码错误、网络结构问题，相较于静态图框架，极大降低了实验调试难度，适合新手开展深度学习实验与算法尝试。

(2) 语法原生简洁，入门门槛低：PyTorch 语法完全贴合 Python 原生编程规范，代码逻辑清晰、可读性强，无冗余复杂语法。实验代码简洁易懂，易于理解神经网络运行逻辑，非常适合深度学习初学者开展基础实验，快速掌握模型搭建、训练、测试的完整流程。

(3) 科研生态完善，实验资源丰富：全球人工智能社区活跃度极高，国内外顶会论文、新型算法、创新模型均优先基于 PyTorch 开源实现。实验过程中可查阅的教程、开源项目、问题解决方案丰富，便于学生复现经典实验、拓展进阶算法实验。

(4) 硬件适配性广，实验兼容性强：原生支持 CUDA GPU 加速运算，同时适配 CPU、国产 AI 芯片、云端算力平台，可满足个人电脑小规模实验、服务器大规模数据集训练等不同实验场景，适配各类实验硬件环境。

(5) 高度自定义，支持创新实验：框架开放性极强，网络层结构、损失函数、训练逻辑、梯度计算规则均可自主定制，不局限于固定模板。除基础验证实验外，可支撑学生开展算法改进、网络结构创新等拓展性创新实验，适配多层次实验教学需求。

更多资料可参考：Pytorch: <https://pytorch.org/>

4.2 Tensorflow

TensorFlow (TF) 是一个采用数据流图 (data flow graphs)，用于数值计算的开源软件库。节点 (Nodes) 在图中表示数学操作，图中的线 (edges) 则表示在节点间相互联系的多维数据数组，即张量 (tensor)。它灵活的架构让你可以在多种平台上展开计算，例如台式计算机中的一个或多个 CPU (或 GPU)，服务器，移动设备等等。

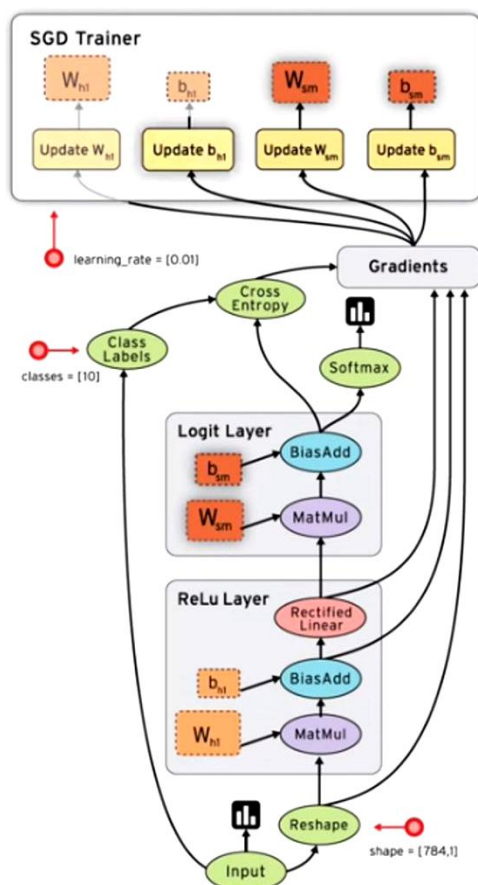


图 4.1 TF 数据流图

4.2.1. 数据流图

数据流图用“结点” (nodes) 和“线” (edges) 的有向图来描述数学计算。“节点”一般用来表示施加的数学操作，但也可以表示数据输入 (feed in) 的起点/输出 (push out) 的终点，或者是读取/写入持久变量 (persistent variable) 的终点。“线”表示“节点”之间的输入/输出关系。这些数据“线”可以输运“size 可动态调整”的多维数据数组，即“张量” (tensor)。张量从图中流过的直观图像是这个工具取名为“Tensorflow”的原因。一旦输入端的所有张量准备好，节点将被分配到各种计算设备完成异步并行地执行运算。如图 4.1 所示：

4.2.2. Tensorflow 的基本使用

使用 TensorFlow 时，应该知道 TensorFlow：

- 使用图(graph)来表示计算任务；
- 在被称之为会话(Session)的上下文(context)中执行图；
- 使用 tensor 表示数据；
- 通过变量(Variable)维护状态；
- 使用 feed 和 fetch 可以为任意的操作(arbitrary operation)赋值或者从其中获取数据。

1. 概述

TensorFlow 是一个编程系统，使用图来表示计算任务。图中的节点被称之为 op (operation)。一个 op 获得 0 个或多个 Tensor，执行计算，产生 0 个或多个 Tensor。每个 Tensor 是一个类型化的多维数组。例如，你可以将一小组图像集表示为一个四维浮点数数组，这四个维度分别是[batch, height, width, channels]。

一个 TensorFlow 图描述了计算的过程。为了进行计算，图必须在会话里被启动。会话将图的 op 分发到诸如 CPU 或 GPU 之类的设备上，同时提供执行 op 的方法。这些方法执行后，将产生的 tensor 返回。

2. 计算图

TensorFlow 程序通常被组织成一个构建阶段和一个执行阶段。在构建阶段，op 的执行步骤被描述成一个图；在执行阶段，使用会话执行执行图中的 op。例如，通常在构建阶段创建一个图来表示和训练神经网络，然后在执行阶段反复执行图中的训练 op。

3. 构建图

构建图的第一步，是创建源 op (source op)。源 op 不需要任何输入，例如常量(Constant)。源 op 的输出被传递给其它 op 做运算。TensorFlow Python 库有一个默认图(default graph)，op 构造器可以为其增加节点，这个默认图对许多程序来说已经足够用了。

4. 神经网络模型搭建 (tf.keras.layers 模块)

TensorFlow 依托内置 Keras 高阶 API 实现快速模型搭建，封装了全套成熟的网络层、激活函数、损失函数，支持序列模型、函数式模型、自定义模型三种搭建方式，适配各类基础深度学习实验。实验常用二维卷积层、池化层、全连接层、归一化层等，代码简洁、规范性强，适合快速搭建标准神经网络。

```
1. import tensorflow as tf
2. from tensorflow.keras import layers, models
3. # 1. 搭建卷积神经网络基础层
4. # 二维卷积层：输入 3 通道，输出 16 通道，卷积核 3*3
5. conv2d = layers.Conv2D(filters=16, kernel_size=3, strides=1, padding="same", activation="relu")
6. # 最大池化层
7. pool = layers.MaxPooling2D(pool_size=2, strides=2)
8. # 全连接层
9. fc = layers.Dense(units=10)
10. # 2. 快速构建序列模型
11. model = models.Sequential([
12.     layers.Conv2D(16, 3, padding="same", activation="relu", input_shape=(32, 32, 3)),
13.     layers.MaxPooling2D(2),
```

```

14.     layers.Flatten(),
15.     layers.Dense(10)])
16. # 3. 常用损失函数
17. ce_loss = tf.keras.losses.SparseCategoricalCrossentropy(from_logits=True)
18. mse_loss = tf.keras.losses.MeanSquaredError()

```

5. 自动梯度计算（tf.GradientTape 模块）

TensorFlow 2.x 依托 tf.GradientTape 梯度磁带机制实现自动微分，支持动态记录运算过程，自动完成链式求导与梯度计算，无需手动推导梯度公式。该模块灵活适配自定义训练逻辑、自定义损失函数实验，是模型反向传播训练的核心基础。

```

1. import tensorflow as tf
2. # 1. 定义可训练变量（模型参数）
3. w = tf.Variable([1.0, 2.0, 3.0])
4. b = tf.Variable([0.5])
5. # 2. 梯度磁带记录运算过程
6. with tf.GradientTape() as tape:
7.     y = tf.reduce_sum(w * 2 + b)
8. # 3. 自动求解梯度
9. grad_w, grad_b = tape.gradient(y, [w, b])
10. print(grad_w.numpy())
11. print(grad_b.numpy())

```

4.2.3. TensorFlow 的特点分析

（1）静态计算图为主，运行效率高、稳定性强：TensorFlow 核心基于静态计算图机制，训练前预先完成整张网络计算图的构建、优化与编译，运行过程中无需动态解析代码，运算速度更快、资源占用更低。在大规模数据集训练、长时间迭代实验中，模型训练稳定性更强，不易出现梯度异常、运算报错等问题，适合开展大批量、高复杂度的深度学习训练实验。

（2）版本迭代成熟，工业生态完备：框架开源时间早、迭代周期长，在工业界落地案例海量，金融、安防、自动驾驶等领域的成熟 AI 项目大多基于 TensorFlow 开发。同时配套完善的官方文档、标准化开发规范、企业级开源项目，适合学生学习标准化、工程化的深度学习开发流程。

（3）TF 2.x 动态静态兼容，实验友好度大幅提升：传统 TensorFlow 1.x 语法繁琐、调试困难，而 TensorFlow 2.x 默认开启动态图机制，兼容 PyTorch 式的即时调试模式，保留了静态图的高效性与动态图的易调试性。既满足新手基础实验调试需求，又能满足高阶工程化训练与部署实验需求，适配多层次教学实验场景。

更多资料可参考：Tensorflow: <https://www.tensorflow.org/>

第五章 上机实验

5.1 实验要求

- 1、在上机前要进行预习。
- 2、实验建议使用 **Python 语言** 完成。
- 3、每次上机完成后要写出实验报告，格式如下：

一. 实验目的
二. 实验内容
三. 实验结果：含所编写的程序及程序的运行结果
四. 实验结果分析

4、首先要完成一个应用程序框架,之后的每一个实验程序都要在该统一框架内运行(如图例 5.1 所示),所有实验做完后,提交一个完整的程序。该 GUI 框架参考代码:

<https://github.com/Dou0405/GUI-Reference-Code-for-the-Intelligent-Image-Processing-Course/tree/main>

该代码基于 PyQt5, 并已预留各实验子界面接口。



图 5.1 操作界面图例

5.2 实验内容

实验一 图像变换实验

1. 实验目的:

掌握图像傅立叶变换、离散余弦变换和哈达玛变换的基本原理及编程实现,并观察其逆变换重建图像的结果。

掌握使用预训练的 bmsbj2018_factorized 深度学习模型对 Lena 图像进行评估、编码压缩与解码重建的过程。

对比并分析基于深度学习的压缩重建结果与传统三种变换重建结果的差异。

2. 实验内容:

对 Lena 或 cameraman 图像进行傅立叶、离散余弦、哈达玛变换。在频域,对比他们的变换后系数矩阵的频谱情况,进一步,通过逆变换观察不同变换下的图像重建质量情况。基于预训练的 bmsbj2018_factorized 权重对 Lena 或 cameraman 图像进行压缩与重建。

3. 实验要求

实验采用灰度图像,该图像每象素由 8 比特表示。具体要求如下:

- (1) 对图像进行傅立叶变换、获得变换后的系数矩阵;
- (2) 将傅立叶变换后系数矩阵的频谱用图像输出,观察频谱;
- (3) 通过设定门限,将系数矩阵中 95%的(小值)系数置为 0,对图像进行反变换,获得逆变换后图像;
- (4) 观察逆变换后图像质量,并比较原始图像与逆变后的 PSNR。
- (5) 对输入图像进行离散余弦、哈达玛变换,重复步骤 1-5;
- (6) 比较三种变换的频谱情况、以及逆变换后图像的质量(PSNR)。
- (7) *选做:基于预训练的 bmsbj2018_factorized 权重对 Lena 或 cameraman 图像进行压缩与重建,评估图片重建质量。具体请参考附录 A。

4. 现场操作:

比较相同图像质量(PSNR)下,步骤(3)中三种变换的系数矩阵,最少需要保留的非零元素个数。一般而言,需要保留的非零元素越少,则将这种变换用于图像编码,越易于节省码率。

比较不同 quality level 的 bmsbj2018_factorized 预训练权重的压缩重建质量,总结压缩指标 bpp 和重建质量 PSNR 之间的规律。

具体要求将在实验现场给出。

实验二 图像增强复原实验

1. 实验目的:

利用空间均值滤波，空间中值滤波以及预训练的 DnCNN 深度学习模型，对降质图像进行复原（去噪），比较不同方法处理后的图像间的差异，以及滤波器大小对结果的影响。

2. 实验内容:

(1) 利用空间均值滤波方法进行图像增强复原;

(2) 利用空间中值滤波方法进行图像增强复原;

(3) 利用深度学习去噪模型 DnCNN 进行图像增强复原。该项内容为本实验的重点与难点。基于代码库: <https://github.com/Dou0405/DnCNN-torch-cpu>.

3. 实验要求:

(1) 输入图像采用 Lena 或 cameraman, 编程实现为输入图像添加高斯噪声与椒盐（脉冲）噪声，程序需要能够指定噪声的均值和方差或两个噪声分量中每一个的概率。

(2) 根据后附的实验指南（附录 B），在本地配置 conda+torch (cpu 版本) 环境；基于给定的代码库，部署预训练好的 DnCNN 模型，并对加噪后的图像进行测试。

(3) *选做：附录 B 进阶探究部分。

(4) 对每一种方法通过计算复原出来的图像的峰值信噪比，比较不同空间掩码的大小的选择对去噪效果的影响。

(5) 将降质图像和利用最优参数恢复后的图像同时显示出来，以便比较。

(6) 实验部分应包括高斯噪声和椒盐噪声的生成过程，原始图像和添加噪声后的图像；加噪图像通过不同方法复原后的结果，并比较复原后的图像之间的差异，另探究滤波器大小对结果的影响。

4. 实验指南：见附录 B。

5. 具体操作将在现场进行讲解。

实验三 CIFAR-10 物体识别

1. 实验目的:

掌握分类、识别问题的实质,了解各种分类问题的机器学习方法,并至少掌握一种,熟悉 Python 编程。

2. 实验内容:

对实验提供含噪声的物体数据库 CIFAR-10 进行训练和测试,最终能够较为准确的识别数据库中的物体。

3. 实验要求:

编写一个完整的 Python 程序,选取一种合适的机器学习方法,对实验提供的物体数据库 CIFAR-10 进行训练和测试,最终能够较为准确的识别数据库中的物体。

CIFAR-10 数据集说明:

本数据及包含了 6 万张分辨率为 32×32 的图片,一共分为了 10 类,分别为:飞机、汽车、鸟、猫、鹿、狗、青蛙、马、船、货车

数据文件:cifar-10-python.tar.gz

解压后包含的文件如下图:

名称	修改日期	类型	大小
batches.meta	2009/3/31 12:45	META 文件	1 KB
data_batch_1	2009/3/31 12:32	文件	30,309 KB
data_batch_2	2009/3/31 12:32	文件	30,308 KB
data_batch_3	2009/3/31 12:32	文件	30,309 KB
data_batch_4	2009/3/31 12:32	文件	30,309 KB
data_batch_5	2009/3/31 12:32	文件	30,309 KB
readme	2009/6/5 4:47	Microsoft Edge ...	1 KB
test_batch	2009/3/31 12:32	文件	30,309 KB

batches.meta 记录了数据集中十个类别的对应信息, data_batch_1 到 data_batch_5 存放了训练样本(共 5 万个样本), test_batch 存放了测试样本(1 万个样本)

数据存储格式:由于数据集中每张图像为 32×32 ,有 RGB3 个通道,一个训练样本对应一行有 $32 \times 32 \times 3 = 3072$ 个值,按照 RGB 通道顺序以及每一通道按照行的顺序已排列好。

数据集官网上提供了 python3 读取 CIFAR-10 的方式,以下函数可以将数据集转化为字典类型:

```
1. def unpickle(file):
2.     import pickle
3.     with open(file, 'rb') as fo:
4.         dict = pickle.load(fo, encoding='bytes')
5.     return dict
```

这里提供一个将 CIFAR-10 转换为图片格式的代码：

```
1. def data_save(load_path, save_path):
2.     label_name = ["airplane",
3.                   "automobile",
4.                   "bird",
5.                   "cat",
6.                   "deer",
7.                   "dog",
8.                   "frog",
9.                   "horse",
10.                  "ship",
11.                  "truck"]
12.
13.     data_dict = unpickle(load_path)
14.
15.     for img_idx, img_data in enumerate(data_dict[b"data"]):
16.
17.         img_name = data_dict[b"filenames"][img_idx]
18.         img_label = data_dict[b"labels"][img_idx]
19.
20.         img_lable_name = label_name[img_label]
21.         img_data = np.reshape(img_data, [3, 32, 32])
22.         img_data = np.transpose(img_data, [1, 2, 0])
23.
24.         if not os.path.exists("{}{}".format(save_path, img_lable_name)):
25.             os.mkdir("{}{}".format(save_path, img_lable_name))
26.         cv2.imwrite("{}{}{}".format(save_path, img_lable_name,
27.                                     img_name.decode("utf-8")), img_data)
```

测试结果及其格式要求：

- 1、该算法的准确度，并且不能低于 60%；
- 2、有明确的结果统计形式。例：通过图表显示准确率，Loss 曲线或测试结果可视化等等。（提示：显示方法可以借助 Matplotlib, Tensorboard, Visdom 等等）

提示：可以选取以下几种方法：

- (1) 深度学习方法：卷积神经网络；
- (2) 逻辑回归算法；
- (3) 支持向量机（SVM）；

实验四 图像分割处理实验

1. 实验目的：

- (1) 了解图像分割的基本原理，并利用图像分割算法进行图像分割处理；
- (2) 掌握数学形态学的基本运算；
- (3) 掌握基于深度学习的图像分割方法（Mask R-CNN）；
- (4) 学会使用常见分割评价指标定量分析结果；
- (5) 培养对比分析传统方法与深度学习方法的能力。

2. 实验内容：

- (1) 利用类间方差阈值算法实现图像的分割处理；
- (2) 利用形态学处理进行处理结果修正；
- (3) 使用预训练 Mask R-CNN 模型进行分割。

3. 实验要求：

- (1) 实验用图如图 5.2 所示；



图 5.2 原始图像

(2) 对输入图像进行平滑处理，以减小噪声对分割处理的影响，比较中值滤波范围取不同值时对图像滤波的效果；

(3) 利用类间方差阈值算法对滤波处理后图像进行分割处理，获取分割图像；

(4) 利用数学形态学中的腐蚀和膨胀运算处理，剔除分割处理结果中的一些细小的残余误分割点，在进行腐蚀和膨胀运算时可采用半径为 r 的圆形结构元素，注意比较选取不同 r 值时的处理结果（ r 分别取 1、3、5）。

(5) 编写一个完整的 Python 程序，选取一种合适的机器学习方法，对实验提供的物体数据库 COCO2017 进行训练和测试，最终能够较为准确的对数据库中的物体进行实例分割，如图 5.3 所示。



图 5.3 实例分割任务示意图

4. 实验任务：

- 测试结果要求

- a. 传统图像分割方法

- (1) 中值滤波效果分析：探究不同滤波窗口大小对图像去噪效果的影响

- (2) 数学形态学处理：对比不同结构元素半径（ r 值）对图像边缘与形状的优化作用

- b. 深度学习分割方法

- (1) 模型测试与可视化：选取合适的深度学习模型，对给定图像进行分割并可视化结果

- (2) 实际场景验证：用手机拍摄采集真实场景图像（一张即可），输入模型进行分割，评估其泛化能力与分割效果

- 加分项：

- a. 验证集性能评估：重点关注 Mask-AP 指标表现

- b. 模型优化方案：通过微调策略提升 Mask-AP 得分

- 现场测评

根据现场给出的参数，对测试数据集的图片加不同强度的噪声，然后再测试算法在测试集上的精确度。画出算法准确度与噪声强度的关系图。

5. 深度分割实验指南：见附录 C。

附录 A: 实验一图像变换实验指南

目标: 在 Windows 电脑上, 用本机 Miniconda3 跑通 CompressAI 的 bmshj2018-factorized 预训练模型, 对 lena.png 完成评估、编码、解码, 并得到重建图像。

本教程对应路径: (windows 中路径为 \, linux 中为 /, 注意自己的系统)

- 代码目录: /path/to/your/code (注: 本教程中为方便表示, 设置为 "E:\Codes\CompressAI")
- 输入图像: /path/to/your/image, 放在代码路径下。 (注: 例如代码路径为 "E:\Codes\CompressAI", 图片路径为 "E:\Codes\CompressAI\results\lena_only_input\lena.png")
- Miniconda3 路径: /path/to/your/miniconda3 (注: 本教程中为方便表示, 设置 miniconda3 目录为 "E:\application\miniconda3")

0. 前期准备

安装 miniconda3;

安装 git bash;

通过 git bash 克隆代码仓库:

```
git clone https://github.com/InterDigitalInc/CompressAI CompressAI
```

准备好实验所需图片, 建议放在代码路径下。

1. 实验目标输出

完成后会得到这几个文件:

```
E:\Codes\CompressAI\results\lena_only_bmshj2018_factorized_q3\metrics.json
E:\Codes\CompressAI\results\lena_only_bmshj2018_factorized_q3\lena-bmshj2018-f
actorized-mse-3-ans.json
E:\Codes\CompressAI\results\lena_only_bmshj2018_factorized_q3\lena_q3.bin
E:\Codes\CompressAI\results\lena_only_bmshj2018_factorized_q3\lena_q3_decoded.
png
```

其中:

metrics.json: 模型在 Lena 单张图上的指标。

lena_q3.bin: 压缩后的码流文件。

lena_q3_decoded.png: 从码流解码出来的重建图像。

2. 配置环境

在 Windows 中打开 Anaconda prompt, 之后所有命令都在 Anaconda prompt 里输入。

先进入代码目录:

```
cd /path/to/your/code
```

创建虚拟环境:

```
conda create -n compressai python=3.10 -y
```

激活环境:

```
conda activate compressai
```

安装 PyTorch CPU 版:

```
pip install torch torchvision --index-url https://download.pytorch.org/whl/cpu
```

安装 CompressAI 运行所需的库:

```
pip install einops matplotlib pandas pybind11 pytorch-msssim scipy tqdm tomli wheel  
torch-geometric
```

CompressAI 的完整压缩/解压流程需要两个 C++ 扩展。由于官方只提供 linux 和 macos 的二进制包, 如果是 windows 系统, 我们需要在环境中内置编译器。安装 MinGW-w64 工具链 (用于编译 C++):

```
conda install --override-channels -c conda-forge m2w64-toolchain libpython -y
```

生成编译链接库, 这一步是将 Python 的运行库转换成 MinGW 能够识别的格式。

生成 .def 文件:

```
gendef "E:\application\miniconda3\envs\compressai\python310.dll"
```

显示如下表示成功:

```
* [E:\application\miniconda3\envs\compressai\python310.dll] Found PE+ image
```

生成 .a 文件:

```
dlltool -d python310.def -D
```

```
"E:\application\miniconda3\envs\compressai\python310.dll" -l libpython310.a
```

显示如下表示成功:

```
dlltool: Path components stripped from dllname,
```

```
'E:\application\miniconda3\envs\compressai\python310.dll'.
```

执行手动编译, 直接调用环境中的 g++ 编译出 Windows 专用的 .pyd 文件。

编译 ans 扩展:

```
g++ -O3 -std=c++17 -shared -I"E:\application\miniconda3\envs\compressai\Include"  
-I"E:\application\miniconda3\envs\compressai\lib\site-packages\pybind11\include"  
-I"E:\Codes\CompressAI\third_party\ryg_rans"  
-I"E:\Codes\CompressAI\compressai\cpp_exts\rans"  
"E:\Codes\CompressAI\compressai\cpp_exts\rans\rans_interface.cpp"  
"E:\Codes\CompressAI\libpython310.a" -o  
"E:\Codes\CompressAI\compressai\ans.cp310-win_amd64.pyd"
```

编译 _CXX 扩展:

```
g++ -O3 -std=c++17 -shared -I"E:\application\miniconda3\envs\compressai\Include"  
-I"E:\application\miniconda3\envs\compressai\lib\site-packages\pybind11\include"  
"E:\Codes\CompressAI\compressai\cpp_exts\ops\ops.cpp"  
"E:\Codes\CompressAI\libpython310.a" -o  
"E:\Codes\CompressAI\compressai\_CXX.cp310-win_amd64.pyd"
```

验证安装结果:

```
set PYTHONPATH= E:\Codes\CompressAI
python -c "import compressai; print('成功安装编码器:',
compressai.available_entropy_coders())"
```

如果没有报错并显示 ['ans'], 如下, 则说明安装成功

```
成功安装编码器: ['ans']
```

3. 运行预训练模型评估

确保在仓库根目录:

```
cd E:\Codes\CompressAI
```

设置 PYTHONPATH, 确保 Python 能找到你刚编译好的 C++ 扩展和源码:

```
set PYTHONPATH=E:\Codes\CompressAI
```

执行评估命令, 第一次运行会自动下载权重:

```
python -m compressai.utils.eval_model pretrained
"E:\Codes\CompressAI\results\lena_only_input" -a bmsbj2018-factorized -m mse -q
3 -d "E:\Codes\CompressAI\results\lena_only_bmsbj2018_factorized_q3" -o metrics
--per-image --verbose
```

参数解释:

pretrained: 使用官方预训练模型。

-a bmsbj2018-factorized: 选择模型结构。

-m mse: 使用 MSE 优化的权重。

-q 3: 使用 **quality 3** 的权重。

--per-image: 保存每张图的单独结果。

第一次运行会自动下载权重:

```
bmsbj2018-factorized-prior-3-5c6f152b.pth.tar
```

默认权重缓存位置:

```
C:\Users\你的用户名\.cache\torch\hub\checkpoints
```

评估完成后, 结果保存在:

```
E:\Codes\CompressAI\results\lena_only_bmsbj2018_factorized_q3\metrics.json
```

4. 对 Lena 做编码

执行编码:

```
python examples\codec.py encode
"E:\Codes\CompressAI\results\lena_only_input\lena.png" --model
bmsbj2018-factorized -m mse -q 3 -o
"E:\Codes\CompressAI\results\lena_only_bmsbj2018_factorized_q3\lena_q3.bin"
```

成功时会看到类似:

```
0.202 bpp | Encoded in 0.30s (model loading: 0.03s)
```

生成文件:

```
E:\Codes\CompressAI\results\lena_only_bmsbj2018_factorized_q3\lena_q3.bin
```

5. 从码流解码得到重建图

执行解码：

```
python examples\codec.py decode
"E:\Codes\CompressAI\results\lena_only_bmshj2018_factorized_q3\lena_q3.bin" -o
"E:\Codes\CompressAI\results\lena_only_bmshj2018_factorized_q3\lena_q3_decoded.
png"
```

Model: bmshj2018-factorized, metric: mse, quality: 3

成功时会看到类似：

Model: bmshj2018-factorized, metric: mse, quality: 3

Decoded in 0.40s (model loading: 0.02s)

生成重建图，保存路径为：

```
E:\Codes\CompressAI\results\lena_only_bmshj2018_factorized_q3\lena_q3_decoded.
png
```

可以直接双击打开，和原始 lena.png 对比。

6. 检查输出文件是否齐全

结果输出路径为

```
E:\Codes\CompressAI\results\lena_only_bmshj2018_factorized_q3
```

应该至少看到：

```
lena-bmshj2018-factorized-mse-3-ans.json
```

```
lena_q3.bin
```

```
lena_q3_decoded.png
```

```
metrics.json
```

7. 额外验证：自己计算重建图 PSNR

注意：这里自己计算的 PSNR 与 eval_model 中的 PSNR 有很小差别，主要来自图像保存、取整和统计方式差异。

8. 总结

本实验使用 CompressAI 官方仓库中的 bmshj2018-factorized 预训练模型，对 Lena 图像进行了学习式图像压缩实验。实验使用 mse 指标下 quality=3 的权重，并采用 ans entropy coder 完成真实编码和解码。最终得到压缩码流 lena_q3.bin 和重建图像 lena_q3_decoded.png。

附录 B：实验二图像增强复原实验指南

实验目的

本实验旨在利用空间均值滤波，空间中值滤波以及预训练的 DnCNN 深度学习模型，对降质图像进行复原（去噪），比较不同方法处理后的图像间的差异，以及滤波器大小对结果的影响。

指南聚焦

本指南重点聚焦该实验的重点与难点，即基于预训练 DnCNN 模型的图像复原方法。最终实现基于 conda+python+torch+cpu 的 DnCNN 推理运行。

深度去噪模型介绍

DnCNN^[10] (Deep Convolutional Neural Network for Image Denoising) 是 TIP 2017 提出的一种经典前馈去噪卷积神经网络。DnCNN 引入了残差学习 (Residual Learning) 和批量归一化 (Batch Normalization, BN) 技术。模型并不直接预测无噪声图像，而是去学习预测图像中的“噪声残差”。最终通过将输入带噪图像减去预测出的噪声，即可得到去噪后的复原图像。DnCNN 既可以针对特定强度的高斯噪声进行训练（如 DnCNN-S 系列），也可以通过混合噪声数据训练实现“盲降噪” (DnCNN-B)，在图像去噪领域具有里程碑意义。

代码库介绍

本实验使用的代码仓库：<https://github.com/Dou0405/DnCNN-torch-cpu>。其中包含了模型与对应的权重，模型在 BSD400 数据集上作了预训练。为适配现代 PyTorch 语法及纯 CPU 运行环境，本实验已对底层代码进行了现代化重构。该实验项目的目录结构与核心文件功能如下：

`models.py`：定义了 DnCNN 网络的结构（包含若干层 Conv+BN+ReLU，最后一层仅为 Conv）。

`dataset.py`：数据加载模块，用于在训练时对图像进行分块 (Patch) 和数据增强（旋转、翻转）。

`utils.py`：工具函数库，包含计算峰值信噪比 (PSNR)、网络权重初始化等底层函数。

`test_custom.py`：（本实验重点运行文件）自定义推理脚本，用于读取我们自己准备的、已经含有特定噪声的降质图像，完成模型推理并将复原后的图像保存至本地。

`logs/`：预训练权重目录。存放了在大型数据集上训练好的参数文件 (`net.pth`)，如 DnCNN-S-25（针对 $\sigma=25$ 噪声的 17 层模型）、DnCNN-B（盲降噪的 20

层模型)。

data/: 存放实验数据集的目录, 实验需要用的两张原图存放在 **data/original** 下。加噪的图像需要创建文件夹保存在 **data** 下。

环境配置

(1) conda 环境配置: 推荐使用 conda 创建 Python 3.9 环境。

```
# 创建虚拟环境
conda create -n dncnn python=3.9 -y

# 激活环境
conda activate dncnn
```

(2) torch 以及其他软件包配置: 该项目依赖以下软件包: torch, torchvision, numpy, scikit-image, opencv-python, h5py。其中 torch、torchvision 和 numpy 之间有较强的版本依赖关系, 需要注意。下面给出一个可靠的版本选择方案:

软件包	推荐版本	说明
PyTorch	2.2.2+cpu	稳定版本。强制指定 CPU Wheel。
Torchvision	0.17.2+cpu	与 Torch 2.2.2 官方严格对应版本。
NumPy	1.26.4	确保与各种科学计算库 (如 OpenCV、h5py) 的二进制接口兼容。
scikit-image	0.22.0	skimage.metrics.peak_signal_noise_ratio
OpenCV-Python	4.9.0.80	负责基础的图像读取与写入, 稳定即可。
h5py	3.10.0	负责 train.h5 和 val.h5 训练数据读写。

命令:

```
pip install torch==2.2.2 torchvision==0.17.2 --index-url https://download.pytorch.org/whl/cpu

pip install numpy==1.26.4 scikit-image==0.22.0 opencv-python==4.9.0.80 h5py==3.10.0
```

运行步骤

(1) 数据准备: 在项目根目录的 data 文件夹中新建一个目录 (例如命名为 test)。将经过空间滤波或人工添加了高斯/椒盐噪声的降质图像 (如 Lena、

Cameraman) 存入该目录。

(2) 推理复原

在终端中运行 `test_custom.py` 进行图像复原。需根据降质图像的噪声类型选择合适的预训练模型和网络层数(`num_of_layers` 参数必须与预训练权重的深度严格对应)。

情形 A: 已知特定强度的高斯噪声 (如 `sigma=25`)

```
python test_custom.py --num_of_layers 17 --logdir logs/DnCNN-S-25 --test_data Custom_Noisy
```

情形 B: 未知强度的高斯噪声或复杂盲降噪

```
python test_custom.py --num_of_layers 20 --logdir logs/DnCNN-B --test_data Custom_Noisy
```

(3) 实验结果分析

程序运行结束后,根目录下会自动生成一个 `results` 文件夹。你可以打开该文件夹,观察深度学习复原后的彩色/灰度图像,并将其与前期实验中使用空间均值滤波、中值滤波处理的图像进行横向对比分析,评估不同方法的去噪效果。

*选做: 进阶探究

该代码库还包含了训练代码、训练和测试可用的其他数据集。同学们可以尝试探究下面几个问题:

- 该模型在 Set12 和 Set68 测试集上表现怎么样?
- 训练 DnCNN, 改变训练集的加噪方式, 探究不同训练加噪方式下, 模型测试时面对不同加噪方式的表现。
- 在盲去噪场景下, 改用不同的网络结构或训练加噪方式, 提升 DnCNN 对 lena 和 cameraman 在高斯噪声 `sigma=25` 时的去噪效果。

附录 C：实验四深度分割指南

实验配置

- 下载压缩包
<https://bhpan.buaa.edu.cn/link/AAB14032EEC767484AA750C7C2677DA659>
文件夹包含：detectron2-main.zip,shutu_rcnn.tar,weights.zip
图像分割任务及标注：val2017.zip, annotations_trainval2017.zip
- 将 detectron2 和 weights 解压，shutu_rcnn 不解压

运行环境（配置好的 docker）

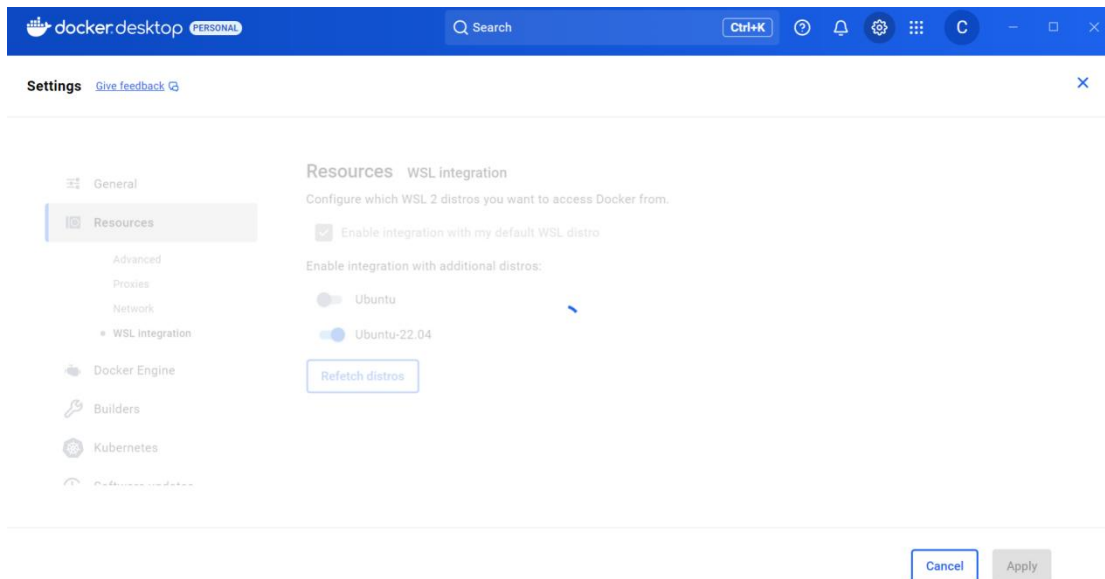
- 安装 WSL
线上方式（如遇操作超时多试几次就好，安装完之后重启）
电脑上按 win+R，输入 cmd

```
1. # 在命令行 cmd 中运行
2. wsl --install #重启
3. wsl --install -d Ubuntu-22.04
4. wsl --set-default-version 2
5. wsl --set-default Ubuntu-22.04
```

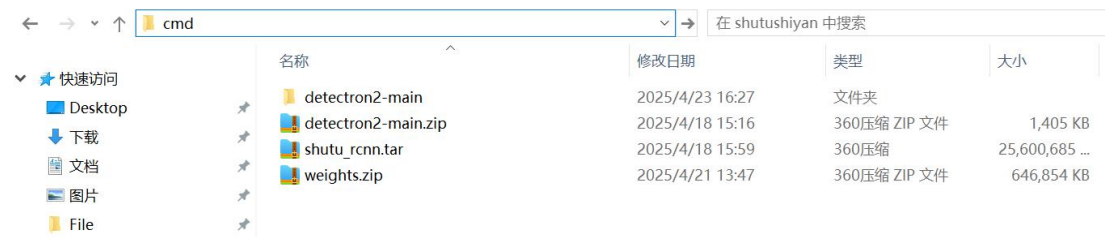
- 配置 NVIDIA Container Toolkit 英伟达驱动，[\[教程\]](#)

```
1. # 进入 WSL-2
2. wsl -d Ubuntu-22.04
3. # 1.添加库（右键粘贴）
4. curl -fsSL https://nvidia.github.io/libnvidia-container/gpgkey | sudo gpg --dearmor -o /usr/share/keyrings/nvidia-container-toolkit-keyring.gpg \
5. && curl -s -L https://nvidia.github.io/libnvidia-container/stable/deb/nvidia-container-toolkit.list | \
6. sed 's#deb https://#deb [signed-by=/usr/share/keyrings/nvidia-container-toolkit-keyring.gpg] https://#g' | \
7. sudo tee /etc/apt/sources.list.d/nvidia-container-toolkit.list
8.
9. # 2.更新 apt
10. sudo apt-get update
11.
12. # 3.安装
13. sudo apt-get install -y nvidia-container-toolkit
```

- 安装 docker
 - 运行[\[Docker Desktop Installer\]](#)，直接在 windows 上进行安装
 - 在设置界面允许 WSL 使用



- 提前下载 shutu_rcnn.tar
- docker 加载镜像 (使用 cmd 运行, 按如下方式在存有压缩包的文件夹中输入并回车打开 cmd)
- detectron2-main 的绝对目录不能有中文, 建议新建文件夹



cmd 中运行:

1. `wsl -d Ubuntu-22.04`
2. # 注意后续的运行都在 wsl 中, 也就是需要首先 `wsl -d Ubuntu-22.04`
3. # 加载 docker image
4. `docker load -i shutu_rcnn.tar`
5. # docker 容器, 在容器的命令行之中完成实验, 注意 detectron2-main 的绝对目录不能有中文
6. `docker run -it --name mask_rcnn -v /mnt/e/detectron2-main:/mnt/data --gpus all shutu_rcnn /bin/bash`
7. # 上面命令的 detectron2-main 在/mnt/XXX(CDE 盘)/XXX 目录下, 自行修改 /mnt 之后的路径为自己 cmd 上显示的路径
8. #例
如: `docker run -it --name mask_rcnn -v /mnt/e/shutushiyang/detectron2-main:/mnt/data --gpus all shutu_rcnn /bin/bash`
9. #进入容器之后
10. `cd /mnt/data`
11. `conda activate rcnn`
- 12.
13. #部分 docker 指令:

```
14. # 退出容器
15. 快捷键: Ctrl+P 之后 Ctrl+Q
16. # 终止容器
17. exit (Ctrl+D)
18. # 启动容器
19. docker start mask_rcnn
20. # 进入容器
21. docker attach mask_rcnn
```

备注：后续的训练、测试环境都在 mask_rcnn 容器中进行

使用预训练模型进行分割

● 运行指令

```
1. cd /mnt/data
2. conda activate rcnn
3. cd demo
4. mkdir results
5. # GPU 运行
6. python demo.py --config-file ../configs/COCO-InstanceSegmentation/mask_rcnn
_R_50_FPN_3x.yaml --input ../datasets/coco/val2017/000000000724.jpg --output resul
ts --opts MODEL.WEIGHTS ../weights/R50_FPN_3x/model_final_f10217.pkl
7. # CPU 运行 (无显卡)
8. python demo.py --config-file ../configs/COCO-InstanceSegmentation/mask_rcnn
_R_50_FPN_3x.yaml --input ../datasets/coco/val2017/000000000724.jpg --output resul
ts --opts MODEL.WEIGHTS ../weights/R50_FPN_3x/model_final_f10217.pkl MODEL.DEVICE
cpu
```

● 得到输出的分割图像在 demo/results 文件夹里面

参考文献

- [1] 夏良正. 数字图像处理 (修订版) [M]. 南京: 东南大学出版社, 2016
- [2] Kenneth R Castleman; 朱志刚等译. 数字图像处理. 北京: 电子工业出版社, 2002
- [3] 张兆礼, 赵春晖, 梅晓丹. 现代图像处理技术及 Matlab 实现. 北京: 人民邮电出版社, 2001
- [4] 张运楚. Python 数字图像处理. 北京: 中国建筑工业, 2021
- [5] Mark Lutz; 刘红伟等译. Python 学习手册. 北京: 机械工业出版社, 2011
- [6] Luciano Ramalho; 安道译. 流畅的 Python. 北京: 人民邮电出版社, 2023
- [7] 王晓华. Opencv+Tensorflow 深度学习与计算机视觉实战. 北京: 清华大学出版社, 2020
- [8] Tariq Rashid; 林赐译. python 神经网络编程. 北京: 人民邮电出版社, 2018
- [9] Michael Nielsen; 朱小虎译. 深入浅出神经网络与深度学习. 北京: 人民邮电出版社, 2020
- [10] Zhang K, Zuo W, Chen Y, et al. Beyond a gaussian denoiser: Residual learning of deep cnn for image denoising[J]. IEEE transactions on image processing, 2017, 26(7): 3142-3155.
- [11] 胡学龙. 数字图像处理 (第 4 版). 北京: 电子工业出版社, 2020